Module 1: Harness Engineering for Coding Agents

Instructions, state, scope, and verification: what each piece assumes about the model, and how to measure it

Course repository `shipharness`

The problem: four failures to check for

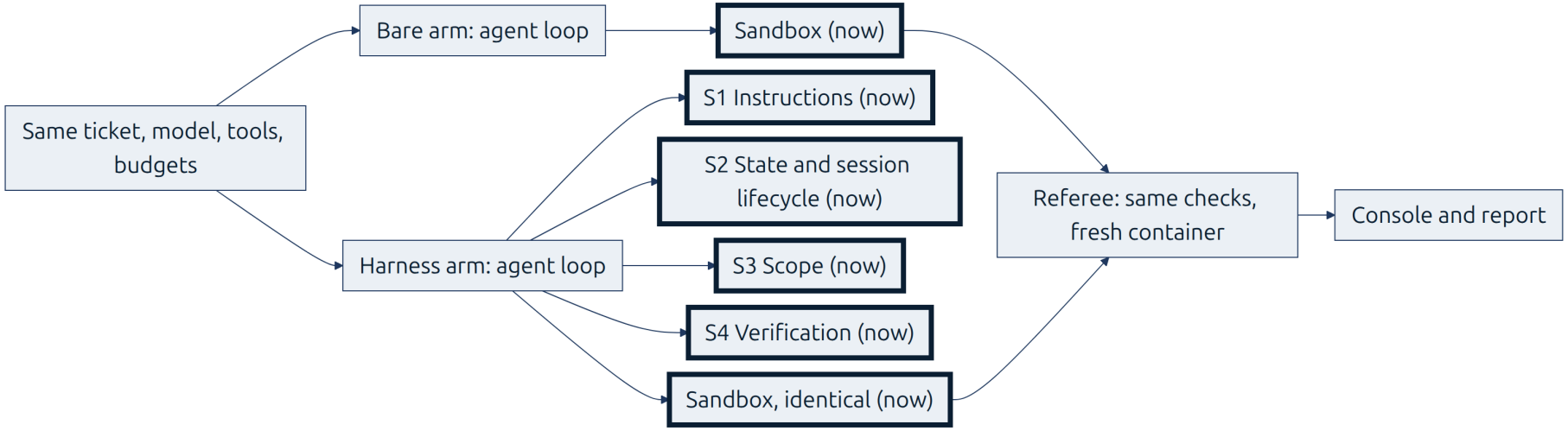
- Cross-tenant data: an Acme user sees Globex issues
- A second session that redoes or breaks the first
- Edits outside the ticket
- The agent's own tests pass while the behaviour is wrong
- The referee checks every run for all four, in both arms

Why it happens: four causes

- "anything it can’t access in-context while running effectively doesn’t exist" (OpenAI, Harness engineering)
- "each new session begins with no memory of what came before." (Anthropic, Effective harnesses)
- Agents copy patterns and take hard-to-undo actions: four sources, side by side (Section 3)
- "agents reliably skew positive when grading their own work" (Anthropic, Harness design)

A harness encodes assumptions about the model

- The work: "design environments, specify intent, and build feedback loops" (OpenAI)
- "every component in a harness encodes an assumption about what the model can't do on its own" (Anthropic, Harness design)
- Assumptions change with the model: the same author later dropped context resets
- Four parts, S1 to S4, plus a sandbox that is the same for both arms
- The course's claim: measure each piece, keep what earns its place



shipharness, TeamTrack, console

- `shipharness`: run an agent with or without the harness, A/B it, judge it
- TeamTrack: a multi-tenant issue tracker with two orgs, Acme and Globex
- Console: five screens on keys `1` to `5`, `F` for full screen per arm

```
def run_and_judge(arm: Arm) -> tuple[AgentRunResult, ScoreRecord]:
    result = run_arm(task, specs[arm], deps, bus, run_dir)
    baseline.join()
    referee.baseline()  # computed by the thread (or now, if the thread failed)
    return result, _judge(referee, specs[arm], result, bus, run_dir)
try:
    with ThreadPoolExecutor(max_workers=2, thread_name_prefix="arm") as pool:
        futures = {arm: pool.submit(run_and_judge, arm) for arm in arms}
        done = {arm: future.result() for arm, future in futures.items()}
finally:
    ...
results = {arm: done[arm][0] for arm in arms}
```

`platform/src/shipharness/ab/demo.py`

How we measure it

- Same agent, model, effort, task, repository, tools, sandbox, turn limit, and cost cap
- One referee judges what ships, the same way for both arms
- House rules R1 to R7: the course's name for OpenAI's "golden principles"
- One primary measure per section, registered before any run; results are counts
- No gap is reported as "no measurable difference on this task with this model"
- One piece at a time: "removing one component at a time" (Anthropic, Harness design)

Check your setup

- Run `make doctor` now
- Run the `harness-setup` assessment on the bare TeamTrack
- Read the gaps it reports: instructions, state, scope, verification

Filled in live: `make doctor` output and the `harness-setup` assessment of the bare TeamTrack

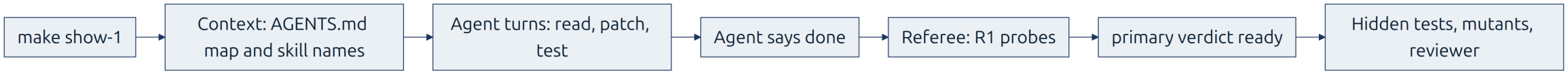
Poll

- F1 org-wide issue search; rule at risk: R1 tenant isolation
- The ticket: a legacy handler without the tenant filter sits next to the right pattern
- Before class, 7 bare-arm runs took this ticket. How many shipped a new R1 violation?
- A) 0-1 B) 2-3 C) 4-5 D) 6-7
- Close the poll before the launch

Filled in live: the room's poll result

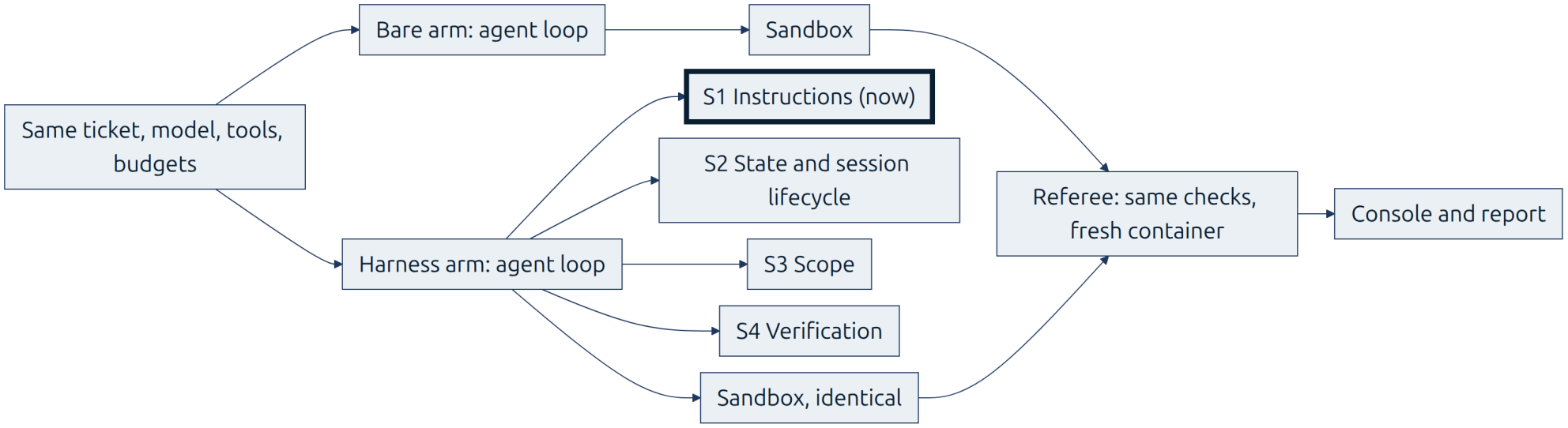
Launching harness in action: F1 org-wide issue search

- Now: run `make show-1` in terminal B; it runs during slides 10-18, back on slide 19
- Harness in action: the harness arm alone on F1, live; not an A/B
- Watch for: which documents does it open before it writes code?
- The referee judges it live: R1 probes first ("primary verdict ready"), then the other checks



S1: what the agent knows

- Why an agent does not apply a rule that lives only outside its context
- Knowledge as a short AGENTS.md map into `docs/`, plus skills that load in tiers
- How to read the S1 result: new R1 violations per run, bare arm against harness arm
- The assumption the map encodes: the model will not apply a rule it has not been pointed to



Problem: the rule is written down, the agent does not apply it

- F1 builds `GET /issues`, an org-wide issue search
- R1: another org's resource is 404, never 403
- The rule is in `docs/HANDBOOK.md`, and both arms can read it
- Nothing is withheld from the bare arm

Cause: what is not in context does not exist

- "anything it can’t access in-context while running effectively doesn’t exist"
- "A giant instruction file crowds out the task, the code, and the relevant docs"
- "Too much guidance becomes non-guidance."
- A long manual goes stale, and no check can keep it true
- Agents copy patterns that already exist, even uneven ones

Harness piece: AGENTS.md as a map

- "roughly 100 lines", "a map", "the table of contents" (OpenAI)
- `docs/` is the system of record
- OpenAI's layout: ARCHITECTURE.md at the root; SECURITY.md, RELIABILITY.md, QUALITY_SCORE.md, PLANS.md in docs/
- Also in docs/: design-docs/, product-specs/, references/

Progressive disclosure

- Docs: "agents start with a small, stable entry point and are taught where to look next" (OpenAI)
- Skills load in three tiers: name and description, then the instructions, then resources
- Tier sizes from the Agent Skills spec: "< 5000 tokens recommended" for the instructions
- SKILL.md needs `name` and `description`

Keep the map true

- Linters and CI jobs check the docs (OpenAI)
- Lint messages that teach: the error carries the fix into the agent's context
- Fairness: the bare arm gets every rule in `docs/HANDBOOK.md`
- A unit test checks that both forms carry each rule id

Code: the map

- The "Where to look" table of TeamTrack's AGENTS.md
- Each row: what you are about to do, and the document to read first

```
## Where to look
| When you are about to | Read first |
|---|---|
| add or change an endpoint | `.agents/skills/add-endpoint/SKILL.md` (step by step) |
| touch a router, service, repository, or model | `ARCHITECTURE.md` (layers, R6) |
| decide who may see or change data | `docs/SECURITY.md` (R1 tenant isolation, R2 roles) |
| shape a list, an error, or a status code | `docs/design-docs/api-conventions.md` (R3) |
| write a create endpoint or any write | `docs/RELIABILITY.md` (R4 idempotency, R5 audit) |
| change the database schema | `docs/RELIABILITY.md` (R7 migrations) |
| change issue or project behaviour | `docs/product-specs/teamtrack-base.md` |
| write or run tests | `docs/QUALITY_SCORE.md` (testing, known gaps) |
| plan a change or check the roadmap | `docs/PLANS.md` |
```

platform/teamtrack/harness/instructions/AGENTS.md

Code: skills, tier 1 only

- `catalogue_block`: one line per skill, name and description
- The full SKILL.md is never loaded up front; the agent reads it when the task matches

```
def catalogue_block(self) -> str:
    skills = self.catalogue()
    if not skills:
        return ""
    rows = [f"- {s.name}: {s.description} (read {s.path}/SKILL.md to use it)" for s in skills]
    return (
        "Available skills (load a skill's SKILL.md only when the task needs it):\n"
        + "\n".join(rows)
    )
```

platform/src/shipharness/runner/skills_loader.py

Code: the stable prefix

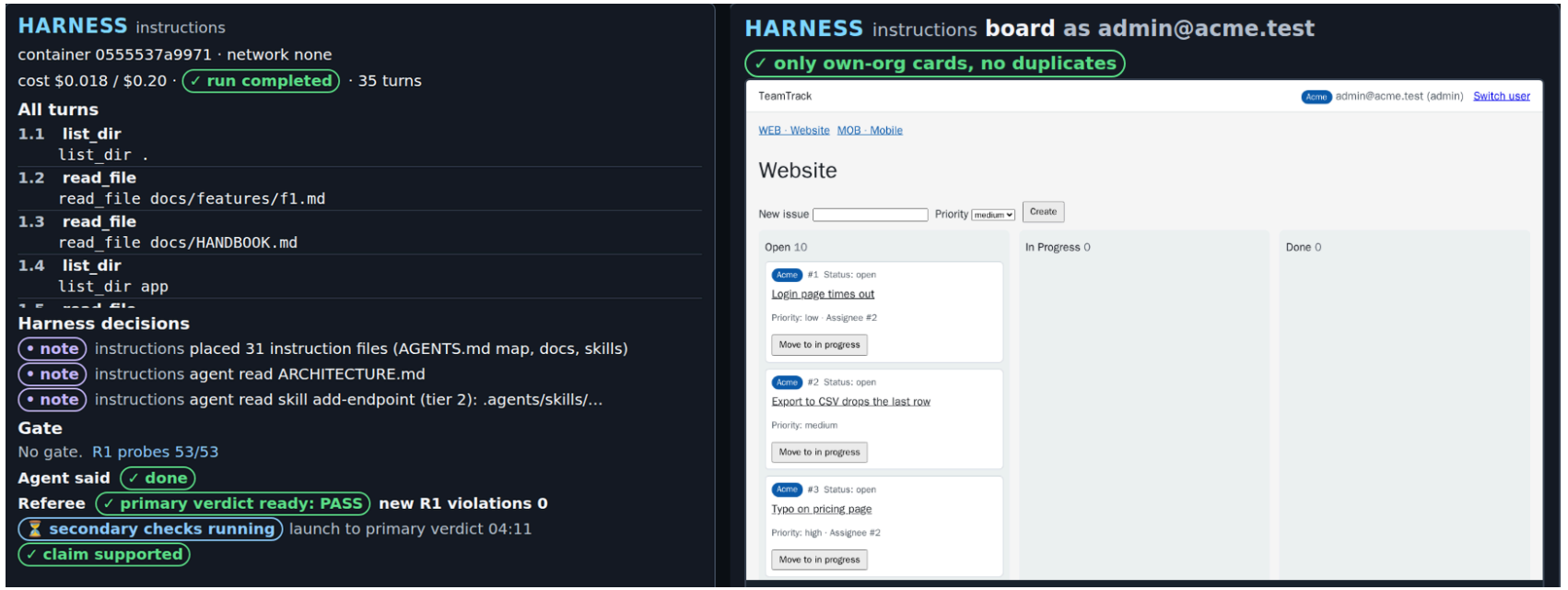
- One stable text block: the map, then tier 1 of every skill
- The documents the map points to are not injected; the agent opens them

```
agents_md = strip_markers((source / MAP_FILE).read_text(encoding="utf-8")).strip()
blocks = [
    "Repository map (AGENTS.md at the workspace root). It points to the documents to read "
    "before you change code; open them with read_file when your task needs them.",
    agents_md,
]
catalogue = SkillsLoader(source / SKILLS_DIR, workspace_prefix=SKILLS_DIR).catalogue_block()
if catalogue:
    blocks.append(catalogue)
return "\n\n".join(blocks)
```

platform/src/shipharness/harness/instructions/router.py

Demo block: what to watch

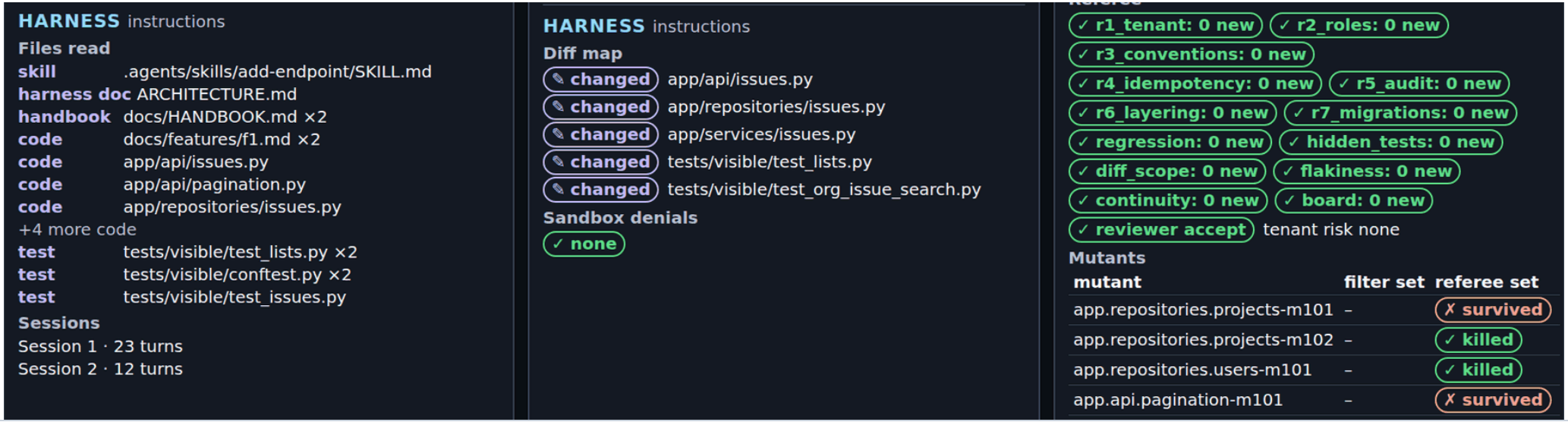
- Screen 1: build turns, test phase, "Agent said done", "primary verdict ready", full verdict
- Screen 3: files read and the diff map; screen 2: the board; screen 5: trace links
- Picture: screens 1 and 2 of a timed run of `make show-1` before class



Filled in live: `make show-1`: the primary verdict and the documents the agent opened

Demo block: reading the result

- The timed run: no new R1 violation; hidden tests 52 of 52; mutants 2 of 4
- One run shows the layer ran, not that it helped
- Before class: bare 0 of 7 failing, harness 0 of 7 (poll answer: A)
- No measurable difference: the bare arm applied R1 without the map
- Picture: screen 3 of the same run



Runbook and next

- Runbook: `01-instructions/demo/README.md`
- Discussion: the map changed nothing on this ticket. What would have to be true for it to earn its place?
- Next: what happens when a session ends halfway

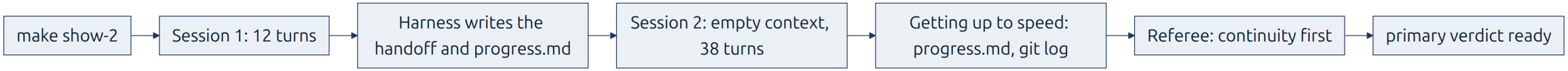
Poll

- F2 comments over two sessions; at risk: continuity and R7
- Before class, both arms stopped after 12 turns and restarted with an empty context
- In those 7 A/Bs, which arms failed the continuity measure?
- A) No arm B) Only bare arms C) Only harness arms D) Some of both
- Close the poll before the launch

Filled in live: the room's poll result

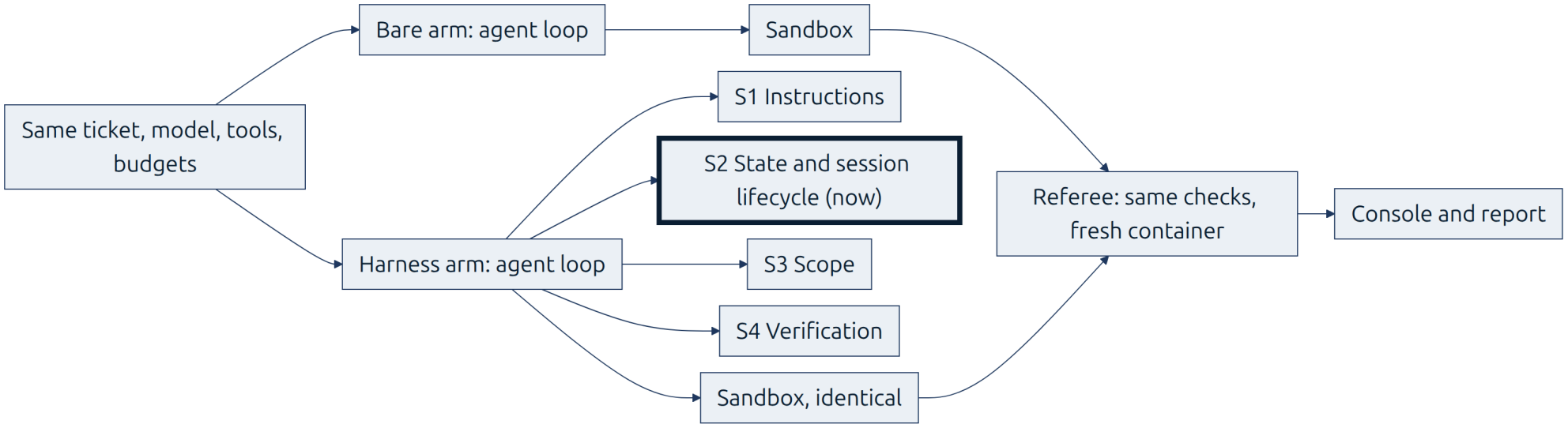
Launching harness in action: F2 comments

- Now: run `make show-2 SESSION_BUDGET=12,38` in terminal B; back on slide 32
- Harness in action: the harness arm alone on F2, first session 12 turns; not an A/B
- Watch for: the handoff written at the end of session 1, and session 2 resuming from it
- Harness arm layers: instructions, state



S2: what the agent remembers

- The initializer and coding agents, `init.sh`, the progress file, and git history
- An exec plan and a structured handoff that let a session resume after a reset
- How to read the S2 result: redone or lost work after a forced stop
- The assumption the state files encode: a fresh session cannot pick up the work from the repository alone



Cause: every session starts empty

- "each new session begins with no memory of what came before." (Anthropic, Effective harnesses)
- "However, compaction isn’t sufficient." (same source)
- What the last session knew must be written down where the next one reads it

Harness piece: state on disk

- An initializer agent and a coding agent (Anthropic, Effective harnesses)
- `init.sh` and a progress file; the course names it `progress.md`
- Git history as state, and as a way back
- In this course, the initializer is a fixed harness step, not an agent

Getting up to speed, and ending a session

- The routine: `pwd`, progress and git log, the feature list, `init.sh`, a basic test
- "End the session by writing a git commit and progress update." (Anthropic, Effective harnesses)
- Both steps are the same in every session

Exec plans and the structured handoff

- Exec plans move from active to completed (OpenAI; OpenAI, PLANS.md)
- A context reset: a fresh agent "combined with a structured handoff" (Anthropic, Harness design)
- The course's mechanism is the session turn budget, the same for both arms
- A reset has a price: "orchestration complexity, token overhead, and latency"
- Later, a newer model "largely removed that behavior", so the author dropped context resets

Code: init.sh

- Migrations first: house rule R7, exactly one head
- Then a basic end-to-end test that stops at the first failure

```
echo "== migrations (house rule R7: exactly one head)"
python -m alembic heads
python -m alembic upgrade head

echo "== basic end-to-end test: issues, stop at the first failure (full suite: run_visible_tests)"
python -m pytest tests/visible/test_issues.py -x -q --no-header -p no:cacheprovider
```

platform/src/shipharness/harness/state/assets/init.sh

Code: stop and resume

- `resume_text`: extra resume text for the harness arm
- The neutral resume sentence comes first, the same for both arms
- A test-phase session starts with the feature's tests

```
def resume_text(summary: str, phase: str = "build") -> str:
    """Extra resume text for the harness arm (the neutral resume sentence comes first)."""
    text = (
        "Start with the Getting up to speed steps before you change code. The structured "
        "handoff from the last session is in session-handoff.md."
    )
    if phase == "test":
        text = f"{text} {TEST_PHASE_FIRST}"
    return f"{text} Its machine summary:\n\n{summary[:3000]}" if summary else text
```

platform/src/shipharness/harness/state/lifecycle.py

Code: the session log

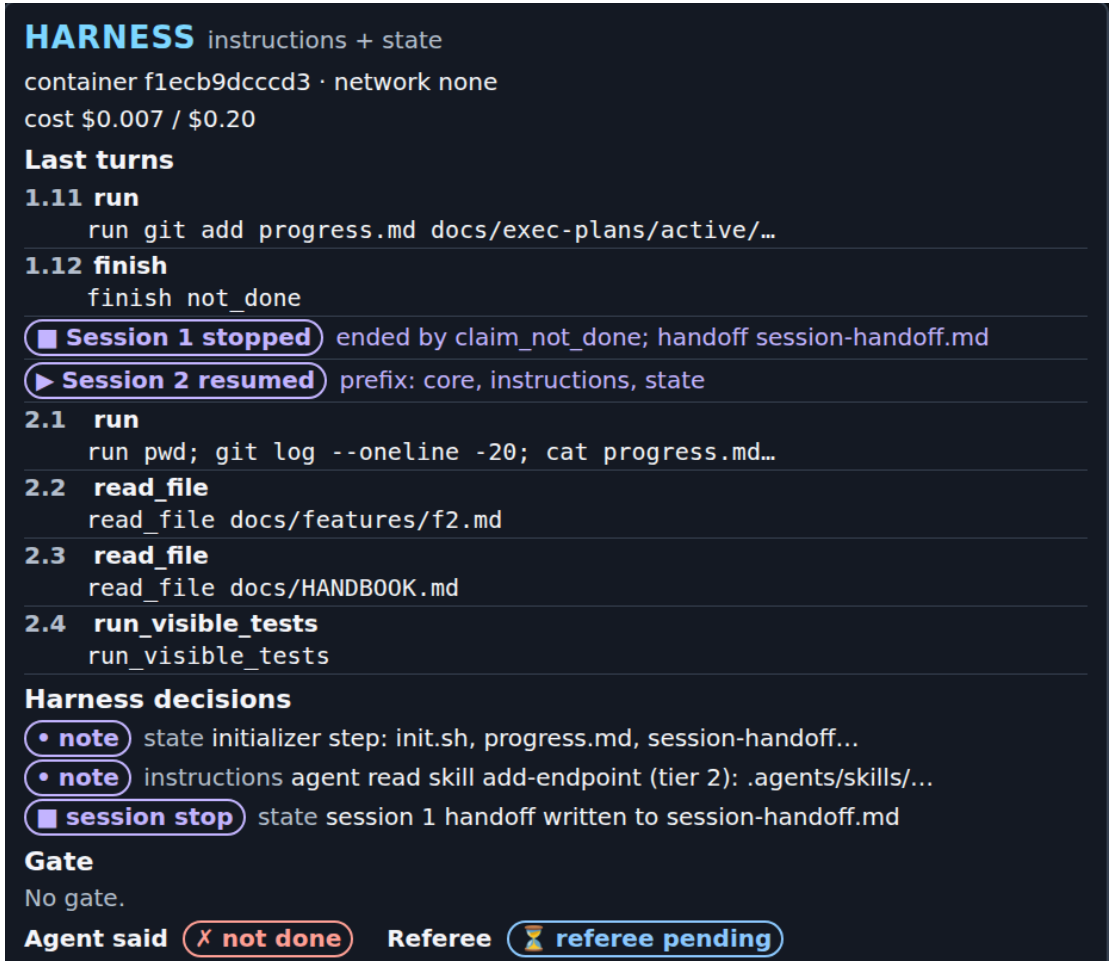
- `session_entry`: one log line per session
- Index, start and end commit, how the session ended, and a summary

```
def session_entry(
    result: SessionResult, files_changed: int, tests: str, claim_summary: str = ""
) -> str:
    """One log line: index, start/end commit, how the session ended, and a summary."""
    entry = (
        f"- Session {result.index} ({result.phase}): start {result.start_sha[:7] or 'none'}, "
        f"end {result.end_sha[:7] or 'none'}, ended by {result.ended_by} after "
        f"{result.turns} turns. {files_changed} files changed; visible tests {tests}."
    )
    summary = " ".join(mask_keys(claim_summary).split())[:200]
    return entry + (f' Agent summary: "{summary}"' if summary else "")
```

`platform/src/shipharness/harness/state/progress.py`

Demo block: what to watch

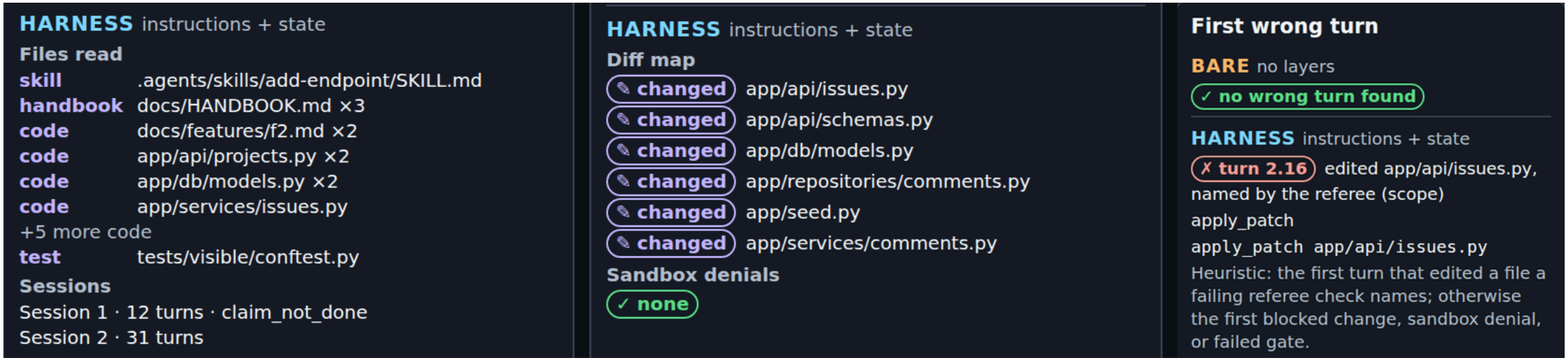
- Screen 1: "■ Session 1 stopped" with the handoff file, then "▶ Session 2 resumed"
- Session 2's first turn: `pwd; git log; cat progress.md`, the getting-up-to-speed routine
- Screen 3: "Sessions", the turns and how each session ended; screen 5: the trace links
- Picture: screen 1 of a timed run of `make show-2` before class



Filled in live: `make show-2`: the primary verdict and the file session 2 read first

Demo block: reading the result

- The timed run: no continuity failure; hidden tests 56 of 56
- First wrong turn (screen 5, a heuristic): turn 2.16 edited `app/api/issues.py`
- Before class: bare 0 of 7 failing, harness 0 of 7 (poll answer: A)
- The measure cannot fail when session 1 builds nothing: 6 of 12 arms did
- Picture: screens 3 and 5 of the same run



Runbook and next

- Runbook: `02-state-lifecycle/demo/README.md`
- Discussion: the bare arm found its own state with `git status`. Which state file would you keep, and how would you measure it?
- Next: a 10-minute break

Break

Back in 10 minutes

Filled in live: return time

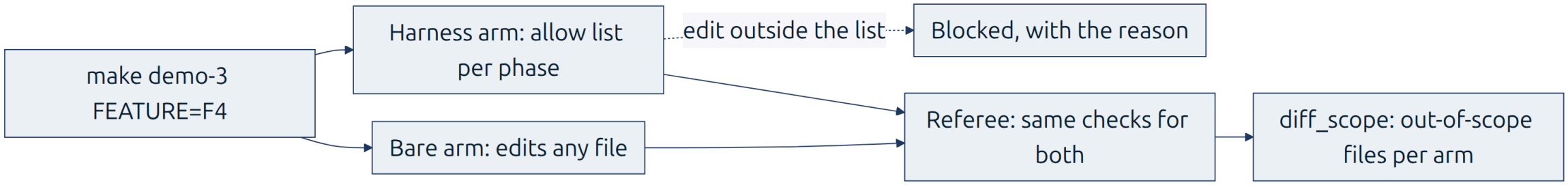
Poll

- F4 optimistic concurrency, tonight's live A/B; at risk: scope
- The build may change the issue, audit, and web files; `app/services/errors.py` is not on the list
- Before class, 3 A/Bs ran this ticket. In how many did the bare arm ship a file outside the list?
- (a) 0 of 3 (b) 1 of 3 (c) 2 of 3 (d) 3 of 3
- Close the poll before the launch

Filled in live: the room's poll result

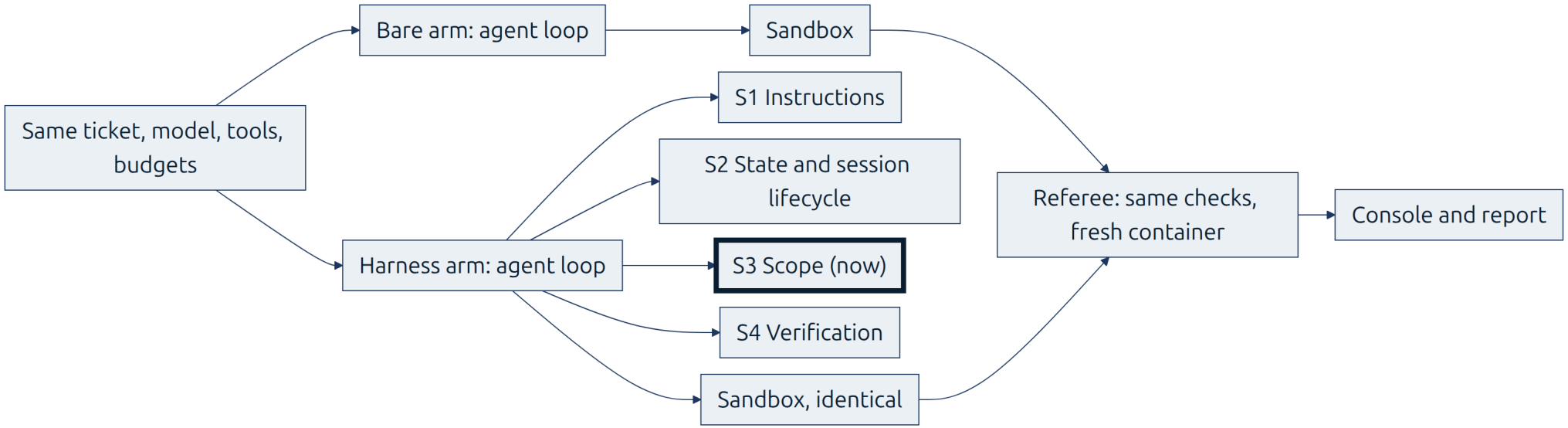
Launching a live A/B: F4, scope

- Now: run `make demo-3 FEATURE=F4` in terminal B; it runs during slides 38-45, back on slide 46
- A live A/B: bare arm and harness arm, same ticket, same limits, at the same time
- Harness arm layers: instructions, state, scope
- Watch for: the bare arm's diff, and the harness arm's blocked edit with its reason



S3: what the agent may change and reach

- One feature at a time, allowed paths per phase, and a `passes` field only the harness sets
- The container limits for both arms, and how to read a `sandbox_denied` event
- How to read the S3 result: out-of-scope files in the shipped diff, bare arm against harness arm
- The assumption the scope layer encodes: the agent will change files outside the ticket unless code stops it



Cause: copied patterns and hard-to-undo actions

- "Codex replicates patterns that already exist in the repository—even uneven or suboptimal ones" (OpenAI)
- "(Run this in a container, not your actual machine)"; the agent once ran `pkill -9 bash` (Anthropic, C compiler)
- File operations "to the project directory only" (Anthropic, Coding agent demo)
- Actions "that are difficult to reverse or affect shared systems" (Anthropic, Prompting best practices)

Harness piece: one feature list, one feature

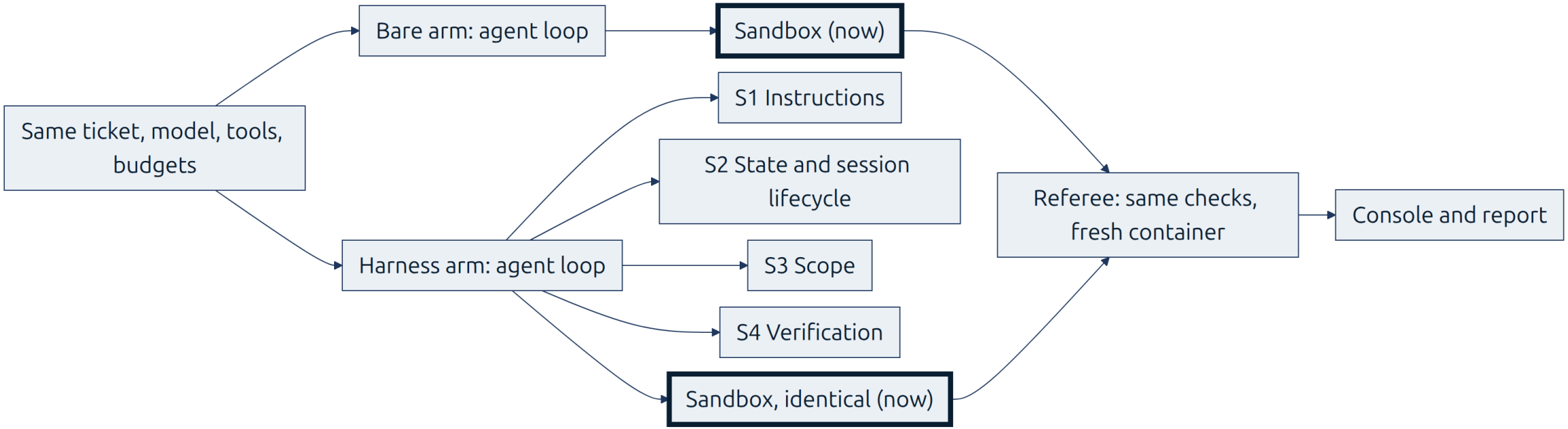
- `feature_list.json` as the source of truth; JSON over Markdown (Anthropic)
- The `passes` field, and one feature at a time
- "It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality."
- In this course, only the harness sets `passes`, with `evidence` and `testedAt`

Allowed paths per phase

- Build and test phases each have an allow list
- A patch outside the list is rejected
- A command's change outside the list is reverted
- The agent reads the reason, which names the phase and the list
- F4's build list leaves out `app/services/errors.py`

Sandbox 1: why a container

- "for each agent, a Docker container is spun up", with no internet access (Anthropic, C compiler)
- A three-layer security model (Anthropic, Coding agent demo)
- "By default, the agent runs with network access turned off." (Codex docs)



Sandbox 2: the same limits for both arms

- `--network none`, memory without swap, 256 processes, non-root
- Read-only root; only `/work` and `/tmp` writable; all capabilities dropped
- `sandbox_denied` events carry a reason code
- Never an A/B variable, so no run can tell whether it earns its place
- Kept on the sources' advice: a container per agent, no internet, project directory only

Code: allowed paths and evidence

- The three `if` checks of `reason()`: harness-only files, state files, the allow list
- `passes` is set only when the evidence is there

```
if rel in HARNESS_ONLY:
    return HARNESS_ONLY[rel]
if self.state_files and any(glob_matches(rel, p) for p in STATE_FILES):
    return None
if self.allowed is None or any(glob_matches(rel, p) for p in self.allowed):
    return None
```

`platform/src/shipharness/harness/scope/guard.py`

```
if evidence is None:
    message = f"passes stays false for {feature['id']}: {why_not}"
    return [HarnessDecision(kind="note", layer="scope", message=message)]
tested_at = now().astimezone(UTC).strftime("%Y-%m-%dT%H:%M:%SZ")
feature.update(passes=True, evidence=evidence, testedAt=tested_at)
```

`platform/src/shipharness/harness/scope/status.py`

Code: the sandbox policy

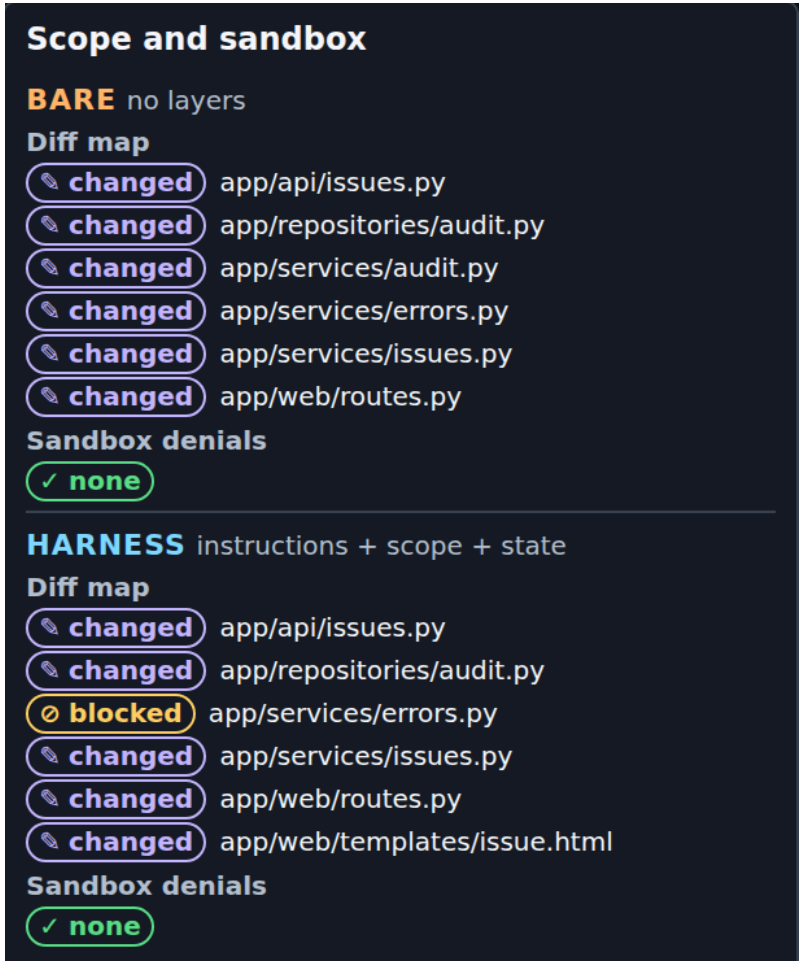
- `SandboxPolicy`: the same for both arms in every run
- Network, CPU, memory, swap, processes, user, root, tmpfs, capabilities

```
network: str = "none"  # only loopback inside the container
cpus: float = 1.0
memory_mb: int = 1024
swap_mb: int = 0  # --memory-swap equal to --memory: no swap
pids_limit: int = 256
user: str = "1000:1000"  # non-root
read_only_root: bool = True  # writes only to /work (workspace) and /tmp
tmpfs: dict[str, str] = {"/tmp": "rw,nosuid,size=256m"}
cap_drop: tuple[str, ...] = ("ALL",)
no_new_privileges: bool = True
```

`platform/src/shipharness/sandbox/policy.py`

Demo block: what to watch

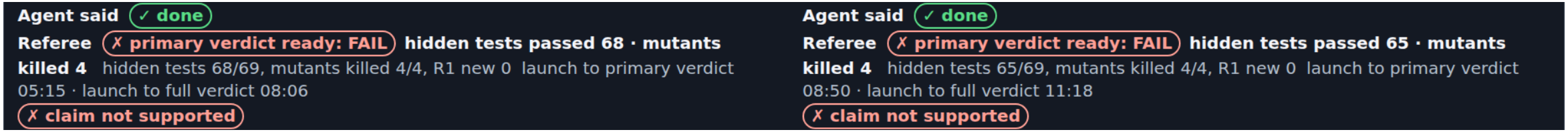
- Screen 3, Scope and sandbox: the bare arm changed `app/services/errors.py`
- The harness arm's diff map shows the same file as ⊘ blocked
- Screen 1: the harness decision ⊘ block, with the reason the agent read
- Screen 3, Verification: `diff_scope` per arm; Sandbox denials for both
- Picture: screen 3 of an S3 confirmation A/B before class



Filled in live: `make demo-3 FEATURE=F4`: each arm's diff map and its `diff_scope` line

Demo block: reading the result

- Before class: in scope, bare 0 of 3, harness 3 of 3 (poll answer: d)
- Harness ahead, by construction: the guard enforces what this measures
- New: on this ticket the bare agent edits a shared file every time
- The pill on screen 1 is F4's own bar, and every arm failed it
- S3's own ticket, F3: no measurable difference, 7 of 7 each
- Picture: screen 1 of the same A/B, the verdict lines



Runbook and next

- Runbook: `03-scope-sandbox/demo/README.md`
- Discussion: the guard won by construction. What would you measure next to it, so a guard that blocks everything cannot look good?
- Next: who decides that the work is done

Poll

- F4 again, now with all four layers; at stake: hidden tests (69) and referee mutants (4)
- A run passes only with all 69 hidden tests passing and all 4 mutants killed
- Before class, 4 A/Bs ran F4 with all four layers in the harness arm. Which arm passed more often?
- A) Harness arm B) Bare arm C) Both, equally often D) Neither arm passed once
- The final feature poll follows on slide 50

Filled in live: the room's poll result

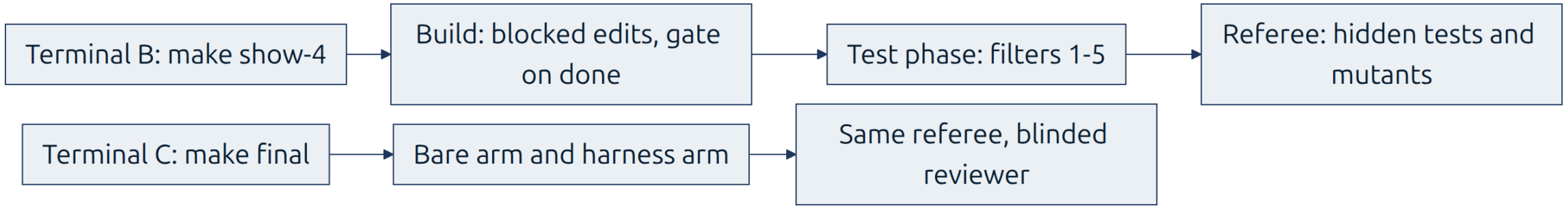
Poll: the final feature

- F5 Bulk close: one session of 30 turns; 46 hidden tests, 3 referee mutants
- F6 Watchers: two sessions, so a context reset; 54 hidden tests, 3 mutants
- F7 Labels: two sessions, so a context reset; 64 hidden tests, 3 mutants
- The winner runs as the final live A/B, and the runner-up after it if the session figure allows
- Close the poll before the launch

Filled in live: the room's poll result

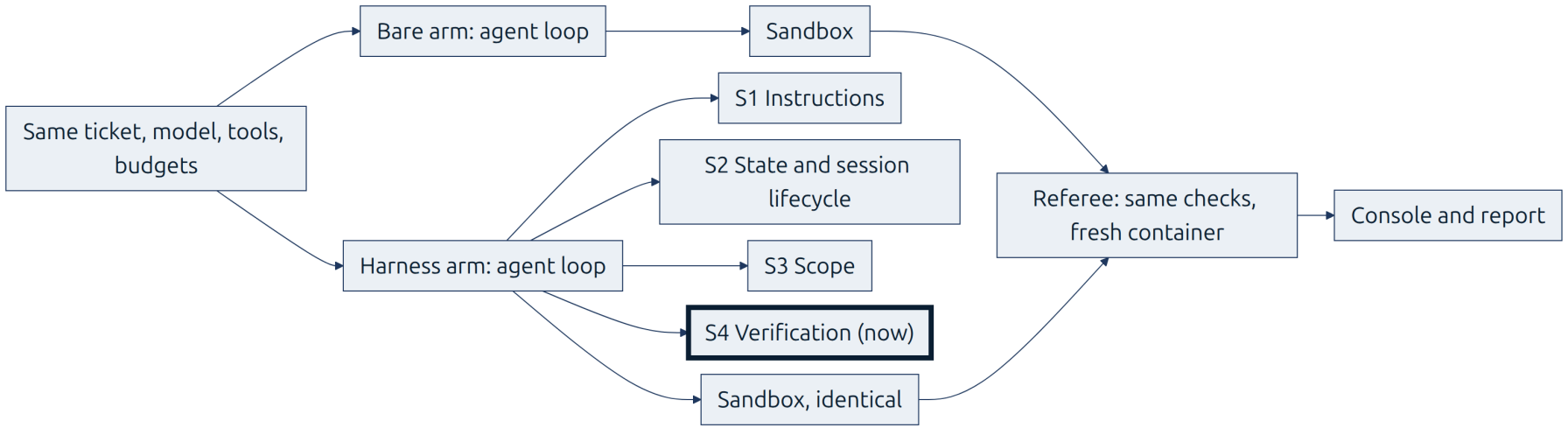
Launching harness in action on F4, and the final A/B

- Now: `make show-4` in terminal B, then `make final FEATURES=<poll winner>` in terminal C
- show-4 runs during slides 52-60, back on slide 61; the final returns on slides 64-67
- show-4: the harness arm alone on F4; watch the gate, the test phase, and the filters
- The final: both arms, all four layers, the blinded reviewer on both diffs



S4: what proves the work is done

- Verification the agent does not control: filters, mutants, house-rule checks, held-out tests, reviewer, gate
- Find the first wrong turn in a trace
- Read a fair A/B: hidden tests passed and mutants caught, as counts per run
- The assumption the gate and filters encode: the agent's "done" and its own tests cannot be trusted



Cause: self-grading and visible tests

- "agents reliably skew positive when grading their own work" (Anthropic, Harness design)
- "while every frontier agent saturates the visible suite, reward hacking persists" (SpecBench)
- Mostly features that pass alone and fail when combined (SpecBench)
- "it is easy to see tests pass and assume the job is done, when this is rarely the case." (Anthropic, C compiler)

Harness piece: test filters

- Filters 1-3 from TestGen-LLM (Meta): builds, passes reliably, adds coverage
- Passes reliably: "every one of five executions"; the referee also re-runs shipped tests five times
- Filters 4 and 5 are the course's own: real assertions, no source change in the test phase
- A test is kept only if it passes every filter

Mutants and hidden tests

- ACH (Meta): mutants for the fault you care about, then tests that catch them
- ACH reports that 49% of tests that uniquely kill a mutant add no line coverage
- Filter-set and referee-set mutants never overlap (course design)
- Hidden tests that combine features (SpecBench)
- 441 hidden tests across F1-F7 and controls; 69 for F4

A separate evaluator and a gate

- A blinded reviewer with grading criteria (Anthropic, Harness design)
- A clean-state gate (Anthropic, Effective harnesses; Anthropic, C compiler)
- House rules R1 to R7, checked by code
- "It is worth the cost when the task sits beyond what the current model does reliably solo." (Anthropic, Harness design)

Measure each piece

- Read the trace: find the first wrong turn; screen 5 shows it per arm, as a heuristic
- Measure one piece: "removing one component at a time and reviewing what impact it had on the final result."
- In shipharness: `make demo-N LAYERS=-<layer>` runs one A/B without that layer
- A section's A/B tests the stack so far; only taking one piece out tests that piece

Code: the filter pipeline

- Each proposed test goes through the filters in order
- The first failure discards it, with a reason; later filters never run

```
for flt in filters:
    try:
        result = flt.evaluate(cand, ctx)
    except Exception as exc:  # noqa: BLE001 - a crashing filter discards, with a reason
    ...
    if not result.reason:
        result = result.model_copy(update={"reason": f"filter {flt.number}: no reason"})
    results.append(result)
    if bus is not None:
        bus.publish("test_filter_result", result, run_id=run_id, arm=arm)
    if not result.passed:
        break  # discarded: later (more expensive) filters never run
```

`platform/src/shipharness/filters/pipeline.py`

Code: a tenant probe and blinding

- `judge`: compare one response with what tenant isolation requires
- A foreign path must return 404; anything else fails

```
def judge(probe: Probe, response: Any, t: Target, home: str, other: str) -> Finding | None:
    """Compare one response with what tenant isolation requires."""
    status, body = response.status_code, json_body(response)
    ok = 200 <= status < 300
    leaked = ok and (
        t.adapter.is_foreign(body, other)
        or (probe.kind == "anonymous" and t.adapter.is_foreign(body, home))
    )
    if probe.kind in ("foreign_path",) or probe.kind.startswith("foreign_param:"):
        failed = status != 404
    else:  # no_path, foreign_body, anonymous: success itself is the failure (no_path: only leaks)
        failed = leaked if probe.kind == "no_path" else ok
```

platform/src/shipharness/checks/r1_tenant.py

Code: the gate

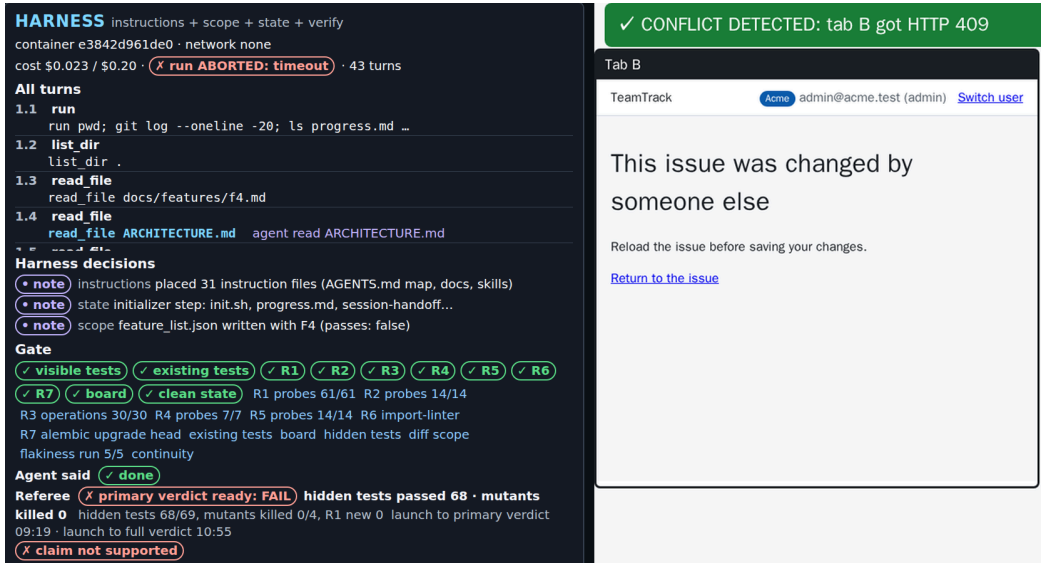
- Verification the agent does not control
- Visible tests, the build-phase checks, the clean-state check
- The gate passes only if no item fails

```
names = ["visible tests", *(_ITEM_ORDER if phase == "build" else []), "clean state"]
...
if phase == "build":
    check_items, check_lines = _check_items(sandbox, cfg, root, on_progress)
    items += sorted(check_items, key=_order)
    lines += check_lines
clean = clean_state_check(
...
passed = not any(i.status == "fail" for i in items)
feedback = "" if passed else _feedback(gate_id, items, lines)
result = GateResult(gate_id=gate_id, items=items, passed=passed, feedback=feedback)
...
    on_update(result)
```

`platform/src/shipharness/harness/verify/gate.py`

Demo block: what to watch

- Screen 1: a ⊘ blocked edit, "done" and the gate pills, then the test phase; screen 2: the conflict panel
- Screen 3: the filter table (kept or discarded, with the reason) and the mutant table
- Picture: screens 1 and 2 of a timed run of `make show-4` before class



Filled in live: `make show-4`: the gate pills and the filter table

Demo block: reading the result

- The timed run: the gate passed; hidden tests 68 of 69, mutants 0 of 4: fail
- The gate never sees the hidden tests or the referee's mutants
- Before class: bare 0 of 4 passing, harness 0 of 4 (poll answer: D)
- One bare arm claimed done with 68 of 69 hidden tests
- Picture: screen 3 of the same run

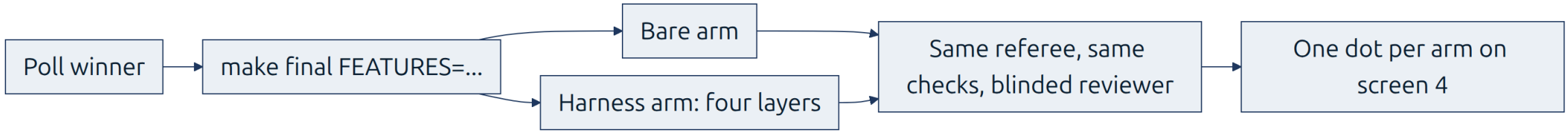


Runbook and next

- Runbook: `04-verification/demo/README.md`
- Discussion: one false "done" in 4 F4 runs per arm, from the bare arm. What would a gate need to catch to earn its turns on this model?
- Next: the final results

Final results: the A/B lines

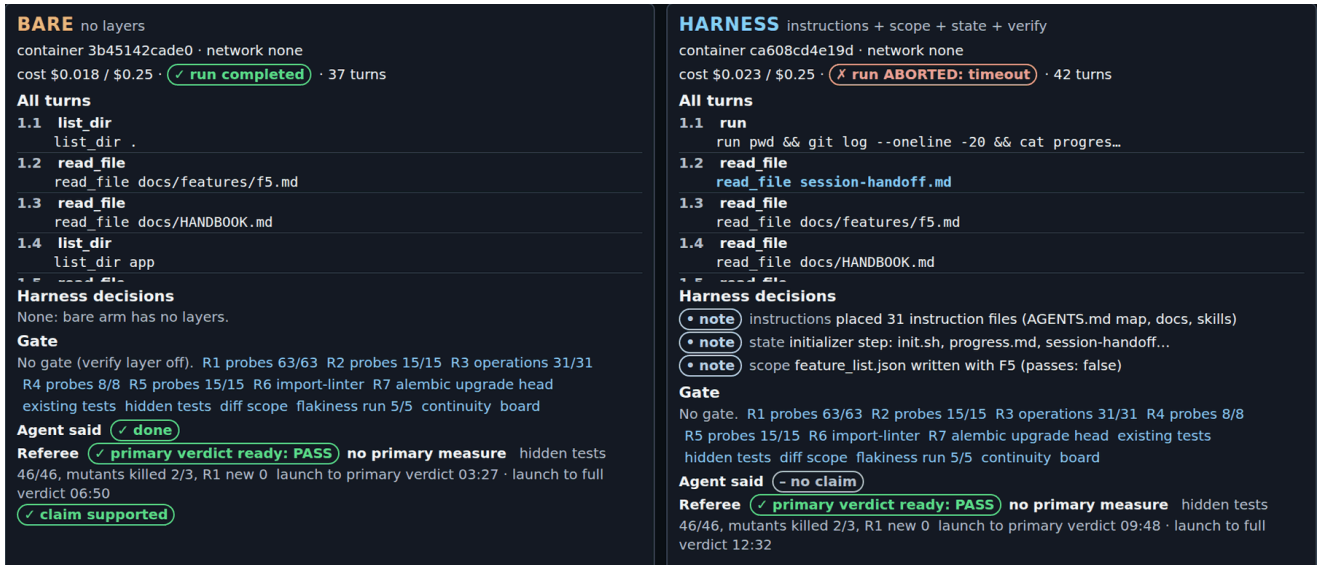
- The poll winner, and the runner-up if it ran; all four layers; class profile
- Run cap $0.25 per arm
- No primary measure registered: read hidden tests and mutants per arm
- One A/B is one dot: read it next to the dots from before class



Filled in live: terminal C's arm lines and session figure

Final results: console

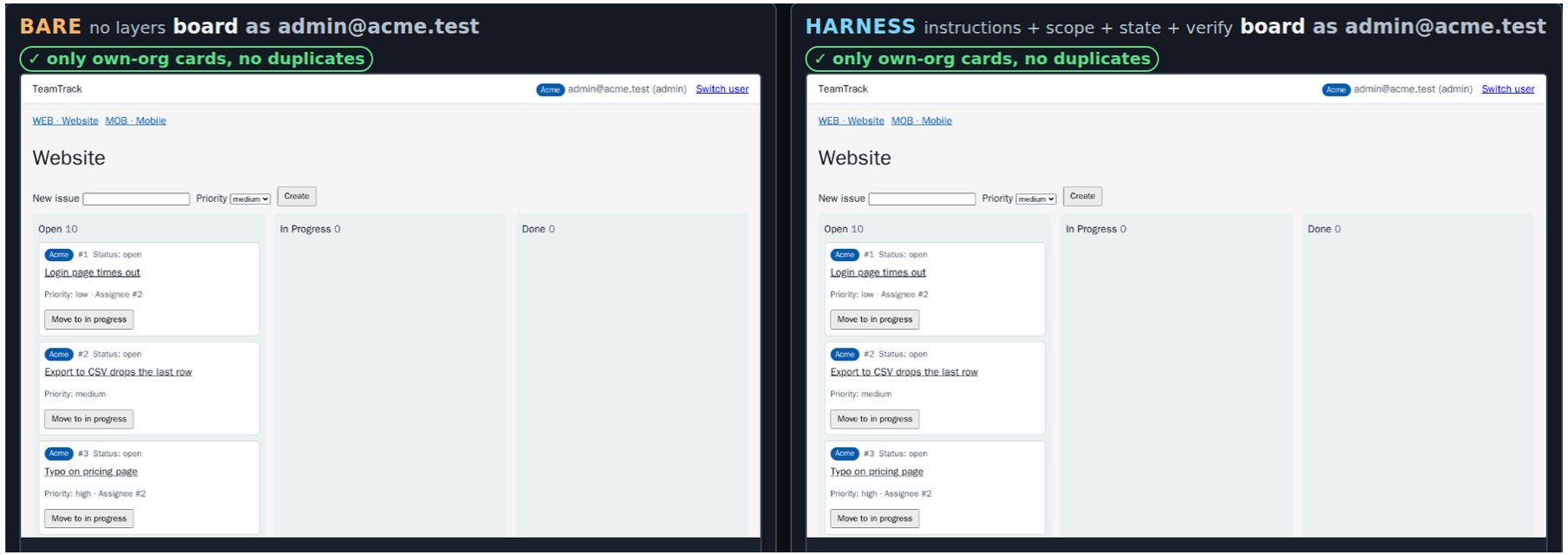
- Screen 1: the turns, harness decisions, gate pills, "primary verdict ready" per arm
- Screen 3: evidence, filters, mutants, reviewer verdict; screen 5: the first wrong turn and the trace links
- Picture: screen 1 of a timed run of `make final FEATURES=F5` before class



Filled in live: screens 1, 3, and 5 of the finished final A/B, both arms

Final results: boards

- Screen 2: each arm's running board as an Acme user
- Look for ⚠ other-org and ⧉ duplicate marks
- Picture: screen 2 of the same timed run



Filled in live: both boards side by side

Final results: screen 4

- Screen 4: one dot per recorded run, wins and ties alike
- Tonight's final adds one dot per arm
- Filled dot = passed, hollow = failed: never colour alone

```
fill = f"var(--{arm})" if dot["passed"] else "var(--surface)"
tip = f"{dot['run_id']}: {dot['value']}" + (
    f" of {dot['total']}" if dot["total"] else ""
)
parts.append(
    f'<circle cx="{x(dot["value"]):.1f}" cy="{y - stack * 10}" r="5" '
    f'fill="{fill}" stroke="var(--{arm})" stroke-width="2">'
    f"<title>{html.escape(tip)} ({dot['status']})</title></circle>"
```

`platform/src/shipharness/ab/report.py`

Filled in live: tonight's final on screen 4

What we measured: every section

Primary measure, runs passing per arm, before class

- S1, F1, no new R1 violation: bare 7 of 7, harness 7 of 7
- S2, F2, no continuity failure: bare 7 of 7, harness 7 of 7
- S3, F3, scope: 7 of 7 each; S3 confirmation on F4, no out-of-scope file: bare 0 of 3, harness 3 of 3
- S4, F4, all hidden tests and mutants: bare 0 of 4, harness 0 of 4
- One harness lead, on scope, by construction; elsewhere no measurable difference on this task with this model
- Post hoc, 34 runs per arm: outside the allowed paths, harness 0, bare 9; done claimed 10 against 19, none false

Assignments

- A1 Instructions: `01-instructions/assignment/`, at most $2.50 and 150 traces
- A2 State and session lifecycle: `02-state-lifecycle/assignment/`, at most $0.40 and 4 traces
- A3 Scope and sandbox: `03-scope-sandbox/assignment/`, at most $0.48 and 4 traces
- A4 Verification and portfolio: `04-verification/assignment/`, at most $0.68
- Each: grading criteria, a starter folder, `make check-assignment-N` offline, one live step, and a full solution
- Total at most $6 and 400 traces; Thursday review

Skills and the starter kit to take home

- `harness-setup`, `assured-tests`, `house-rules`, `skill-template`, in the Agent Skills format
- `uv run shipharness init --target <repo>`: the kit, with TeamTrack's values and REPLACE markers
- `uv run shipharness init --check <repo>` lists the markers still to fill; no model calls
- Guide: `use-in-your-repo/apply-to-your-repo.md`, ending with which layers earn their place in your repository

```
name: skill-template
description: >
  Starting point for writing a new Agent Skill: frontmatter with the spec's fields and a guide to
  the sections of the body. Use when creating a new skill: copy this folder, rename it, set name to
  the new folder name, and replace every placeholder.
license: MIT
```

`use-in-your-repo/skills/skill-template/SKILL.md`

References (1 of 2)

- OpenAI, Harness engineering: leveraging Codex in an agent-first world · Ryan Lopopolo · 2026-02-11
- Anthropic, Effective harnesses for long-running agents · Justin Young · 2025-11-26
- Anthropic, Harness design for long-running application development · Prithvi Rajasekaran · 2026-03-24
- Anthropic, Building a C compiler with a team of parallel Claudes · Nicholas Carlini · 2026-02-05
- Anthropic, Autonomous Coding Agent Demo (README) · 2025-11-26
- Anthropic, Prompting best practices (living page); OpenAI, Using PLANS.md · Aaron Friel · 2025-10-07

References (2 of 2)

- Agent Skills: Specification; What are skills?; How to add skills support · agentskills.io
- The skills-ref validator; Claude Code, Codex, and Cursor skills docs
- TestGen-LLM · Alshahwan et al., Meta · 2024
- ACH, mutation-guided test generation · Foster et al., Meta · 2025
- SpecBench · Zhao, Srikanth, Wu, Jiang (Weco AI) · 2026, and its repository
- Codex docs on sandboxing and network access